

# Unidad 9 - Persistencia: ficheros y acceso a bases de datos

1º CFGS Desarrollo de Aplicaciones Web

Curso 2022/2023

José Miguel Blázquez

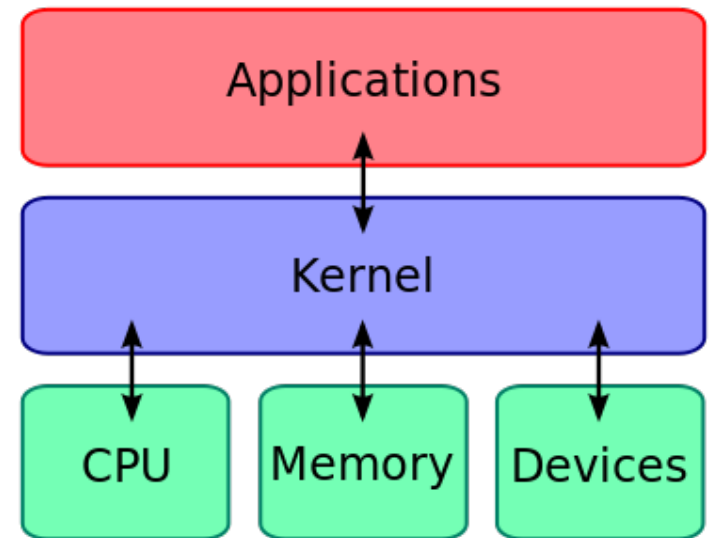
Departamento de Informática

# Persistencia

- Aplicaciones pierden memoria al cerrarse.
- Persistencia: que los datos perduren en el tiempo de forma permanente.
- Datos sobreviven a una ejecución, incluso a la ausencia de energía (volatilidad del medio).
- Se consigue mediante 2 vías:
  - Almacenamiento directo en ficheros
  - Acceso a SGBD

# Persistencia

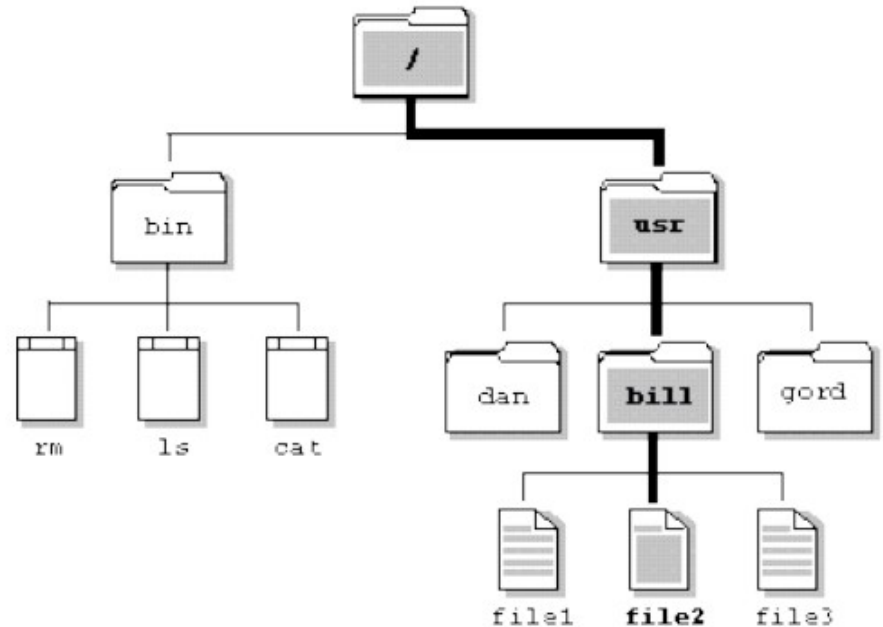
- Memoria principal vs memoria secundaria.
- SO: control del acceso al sistema de archivos.
  - Ofrece mecanismos (capa de abstracción de hardware) normalizados a las aplicaciones para acceso a dispositivos almacenamiento secundario.
- Las librerías permiten despreocuparse de los detalles técnicos del hardware:
  - java.io
  - File
  - FileReader
  - FileWriter
- O de la complejidad del SGBD...
  - JDBC



# Gestión de ficheros

- **Tipos:**

- **Fichero estándar:** contiene cualquier tipo de datos.
- **Directorio o carpeta:** contiene otros ficheros. Permite organizar de forma **jerárquica** los diferentes ficheros. [Es un fichero...]
- **Ficheros especiales:** control de periféricos...

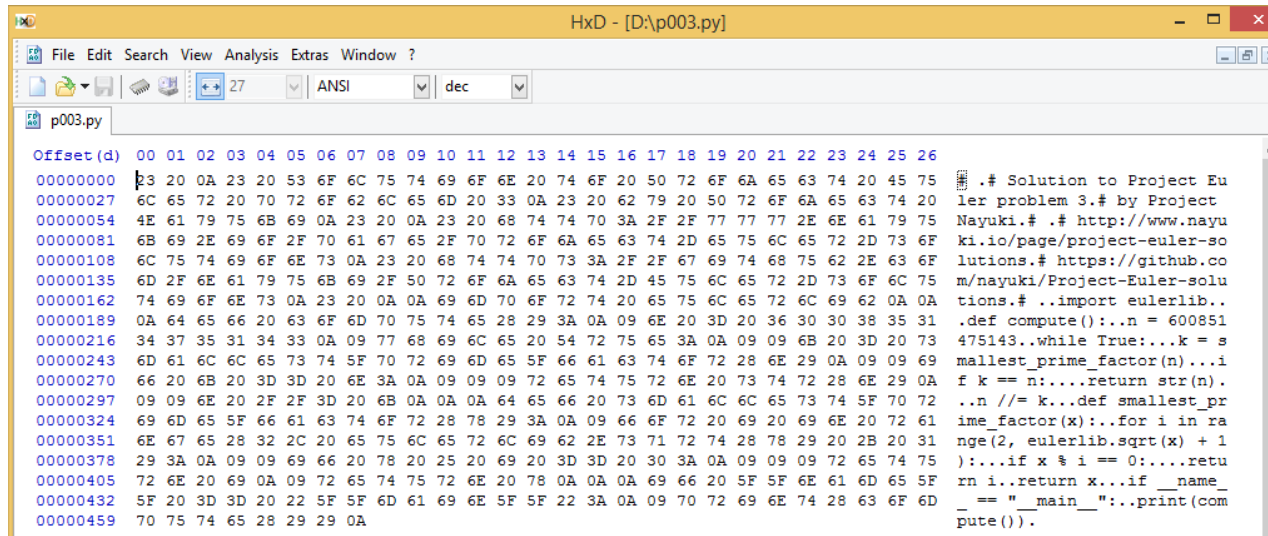


**Parámetros:** nombre, ruta, extensión...

¡Organización jerárquica!

**Ejercicio:** localiza en el **árbol** de directorios uno de tus proyectos de NetBeans, desde la ventana del explorador de archivos. Ahora, abre una consola y navega hasta él...

# Tipos de ficheros según contenido

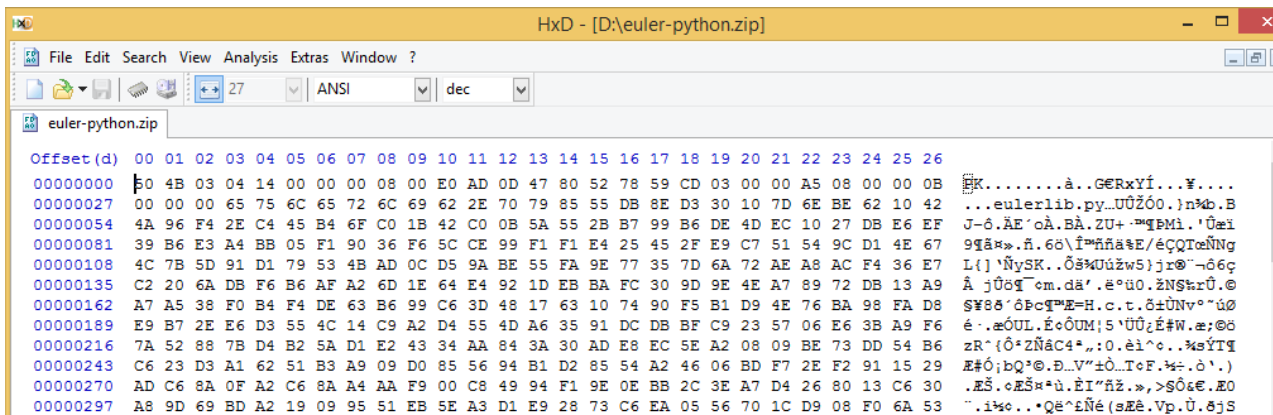


```
Offset(d) 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26
00000000 63 20 0A 23 20 53 6F 6C 75 74 69 6F 6E 20 74 6F 20 50 72 6F 6A 65 63 74 20 45 75
00000007 6C 65 72 20 70 72 6F 62 6C 65 6D 20 33 0A 23 20 62 79 20 50 72 6F 6A 65 63 74 20
00000054 4E 61 79 75 6B 69 0A 23 20 0A 23 20 68 74 74 70 3A 2F 2F 77 77 77 2E 6E 61 79 75
00000081 6B 69 2E 69 6F 2F 70 61 67 65 2F 70 72 6F 6A 65 63 74 2D 65 75 6C 65 72 2D 73 6F
00000108 6C 75 74 69 6F 6E 73 0A 23 20 68 74 74 70 73 3A 2F 2F 67 69 74 68 75 62 2E 63 6F
00000135 6D 2F 6E 61 79 75 6B 69 2F 50 72 6F 6A 65 63 74 2D 45 75 6C 65 72 2D 73 6F 6C 75
00000162 74 69 6F 6E 73 0A 23 20 0A 0A 69 6D 70 6F 72 74 20 65 75 6C 65 72 6C 69 62 0A 0A
00000189 0A 64 65 66 20 63 6F 6D 70 75 74 65 28 29 3A 0A 09 6E 20 3D 20 36 30 30 38 35 31
00000216 34 37 35 31 34 33 0A 09 77 68 69 6C 65 20 54 72 75 65 3A 0A 09 6B 20 3D 20 73
00000243 6D 61 6C 6C 65 73 74 5F 70 72 69 6D 65 5F 66 61 63 74 6F 72 28 6E 29 0A 09 69
00000270 66 20 6B 20 3D 3D 20 6E 3A 0A 09 09 09 72 65 74 75 72 6E 20 73 74 72 28 6E 29 0A
00000297 09 09 6E 20 2F 2F 3D 20 6B 0A 0A 64 65 66 20 73 6D 61 6C 6C 65 73 74 5F 70 72
00000324 69 6D 65 5F 66 61 63 74 6F 72 28 78 29 3A 0A 09 66 6F 72 20 69 20 69 6E 20 72 61
00000351 6E 67 65 28 32 2C 20 65 75 6C 65 72 6C 69 62 2E 73 71 72 74 28 78 29 20 2B 20 31
00000378 29 3A 0A 09 09 69 66 20 78 20 25 20 69 20 3D 3D 20 30 3A 0A 09 09 72 65 74 75
00000405 72 6E 20 69 0A 09 72 65 74 75 72 6E 20 78 0A 0A 0A 69 66 20 5F 5F 6E 61 6D 65 5F
00000432 5F 20 3D 3D 20 22 5F 5F 6D 61 69 6E 5F 5F 22 3A 0A 09 70 72 69 6E 74 28 63 6F 6D
00000459 70 75 74 65 28 29 29 0A
```

```
#!/usr/bin/env python3
# Solution to Project Euler problem 3. # by Project Nayuki. # http://www.nayuki.io/page/project-euler-solutions. # https://github.com/nayuki/Project-Euler-solutions. # ..import eulerlib..
def compute():
    n = 600851475143
    while True:
        k = smallest_prime_factor(n)
        if k == n:
            return str(n)
        n //= k
    def smallest_prime_factor(x):
        for i in range(2, eulerlib.sqrt(x) + 1):
            if x % i == 0:
                return i
        return x
    if __name__ == '__main__':
        print(compute())
```

**Fichero de texto:** los bytes representan sólo caracteres. Puede crearse y visualizarse usando un editor de textos.

TXT, XML, HTML, JSON



```
Offset(d) 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26
00000000 4B 03 04 14 00 00 00 08 00 E0 AD 0D 47 80 52 78 59 CD 03 00 00 A5 08 00 00 0B
00000007 00 00 00 65 75 6C 65 72 6C 69 62 2E 70 79 85 55 DB 8E D3 30 10 7D 6E BE 62 10 42
00000054 4A 96 F4 2E C4 45 B4 6F C0 1B 42 C0 0B 5A 55 2B B7 99 B6 DE 4D EC 10 27 DB E6 EF
00000081 39 B6 E3 A4 BB 05 F1 90 36 F6 5C CE 99 F1 F1 E4 25 45 2F E9 C7 51 54 9C D1 4E 67
00000108 4C 7B 5D 91 D1 79 53 4B AD 0C D5 9A BE 55 FA 9E 77 35 7D 6A 72 AE A8 AC F4 36 E7
00000135 C2 20 6A DB F6 B6 AF A2 6D 1E 64 E4 92 1D EB BA FC 30 9D 9E 4E A7 89 72 DB 13 A9
00000162 A7 A5 38 F0 B4 F4 DE 63 B6 99 C6 3D 48 17 63 10 74 90 F5 B1 D9 4E 76 BA 98 FA D8
00000189 E9 B7 2E E6 D3 55 4C 14 C9 A2 D4 55 4D A6 35 91 DC DB BF C9 23 57 06 E6 3B A9 F6
00000216 7A 52 88 7B D4 B2 5A D1 E2 43 D4 A8 84 3A 30 AD E8 EC 5E A2 08 09 BE 73 DD 54 B6
00000243 C6 23 D3 A1 62 51 B3 A9 09 D0 85 56 94 B1 D2 85 54 A2 46 06 BD F7 2E F2 91 15 29
00000270 AD C6 8A 0F A2 C6 8A A4 AA F9 00 C8 49 94 F1 9E 0E BB 2C 3E A7 D4 26 80 13 C6 30
00000297 A8 9D 69 BD A2 19 09 95 51 EB 5E A3 D1 E9 28 73 C6 EA 05 56 70 1C D9 08 F0 6A 53
```

```
.....à.GERxYí...Y....
...eulerlib.py..UÛÓo.)n%b.B
J-ó.ÀE'óA.BA.ZU+..m%BMi.'Uæi
9qã»..ñ.6ó\î»ññ&E/éçQTœNNg
L{)'NySK..ÔšMÚúzw5}jr@'-ó6ç
Å jÛôq'om.dâ'.è'ú0.ZNškrÛ.©
$W8ó'óBçqME=H.c.t.ô±UNv°'úó
é..æÓUL.éóÓUM;5'UÛ¿E#W.æ;@ó
zR' (Ô'Zñ&C4*,:0.èi^c..%šYTq
E#ó;bQ?@.D..V''±Ô..TóF.%÷.ó.)
.ES.óEŠ»'ù.ÈI'ñž..»>Sô&E.E0
..i%o...Qè^Eñ&E(sE&E.Vp.Û.éjS
```

**Fichero binario:** sus bytes no representan caracteres, pueden representar otros datos: números, imágenes, sonido, etc.

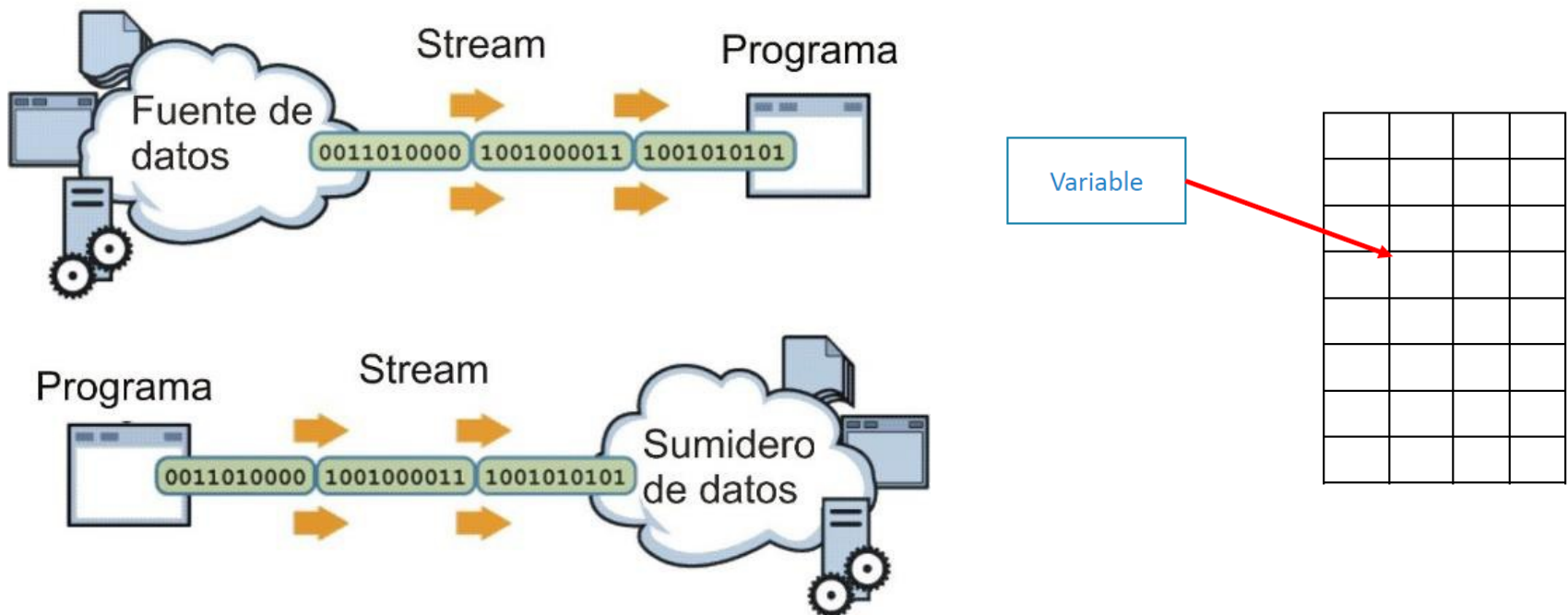
¿editor de textos?

PDF, JPG, DOCX, MP4

# Streams de datos

Para poder realizar las operaciones de lectura y escritura de ficheros, Java establece lo que se conoce como un Stream\* de datos:

- Vía de comunicación entre programa y fichero que permite “moverse” por las distintas partes del fichero
- Existe un “puntero” que señala a las partes del fichero



\* Podemos entender el concepto “Stream” como si de una tubería se tratase...o un canal de comunicación.

# Clase File

```
import java.io.File;
```



La clase File representa una ruta dentro del sistema de archivos.

```
File f = new File (String ruta);
```

UNIX: /usr/bin  
WINDOWS: C:\Windows\System32

**Posible pérdida de multiplataformidad...¿porqué?**

```
File carpetaFotos = new File("C:/Fotos");  
File unaFoto = new File("C:/Fotos/Foto1.png");  
File otraFoto = new File("C:/Fotos/Foto2.png");
```

¿Sabías que...? Una carpeta también es, físicamente, un fichero...

# Rutas absolutas y relativas



Una **ruta absoluta** es aquella que se refiere a un elemento a partir del **raíz** del sistema de ficheros. Por ejemplo "C:/Fotos/Foto1.png"

## ¿Problemas?



Una **ruta relativa** es aquella que **no incluye el raíz** y por ello se considera que **parte desde el directorio de trabajo** de la aplicación. Esta carpeta puede ser diferente cada vez que se ejecuta el programa.

```
File f = new File ("Unidad11/apartado1/Actividades.txt");
```

| Directorio de trabajo | Ruta real                                            |
|-----------------------|------------------------------------------------------|
| C:/Proyectos/Java     | C:/Proyectos/Java/Unidad11/apartado1/Actividades.txt |
| X:/Unidades           | X:/Unidades/Unidad11/apartado1/Actividades.txt       |
| /Programas            | /Programas/Unidad11/apartado1/Actividades.txt        |

**Las rutas relativas pueden resolver el problema de compatibilidad entre plataformas...**

```
File f = new File ("Clientes.txt");
```



# Métodos: ruta de acceso

```
import java.io.File;

public class PruebasFicheros {

    public static void main(String[] args) {
        // Dos rutas absolutas
        File carpetaAbs = new File("/home/josemi/fotos");
        File archivoAbs = new File("/home/josemi/fotos/albania1.jpg");

        // Dos rutas relativas
        File carpetaRel = new File("trabajos");
        File fitxerRel = new File("trabajos/documento.txt");

        // Mostramos sus rutas
        mostrarRutas(carpetaAbs);
        mostrarRutas(archivoAbs);
        mostrarRutas(carpetaRel);
        mostrarRutas(fitxerRel);
    }

    public static void mostrarRutas(File f) {
        System.out.println("getParent()      : " + f.getParent());
        System.out.println("getName()      : " + f.getName() + "\n");
        System.out.println("getAbsolutePath(): " + f.getAbsolutePath());
    }
}
```



```
archivoAbs.close();
...
```

```
+ f.getParent();
+ f.getName() + "\n";
+ f.getAbsolutePath();
```

# Métodos: comprobaciones estado

```
public static void main(String[] args) {  
    File temp = new File("C:/Temp");  
    File fotos = new File("C:/Temp/Fotos");  
    File document = new File("C:/Temp/Documento.txt");  
    System.out.println(temp.getAbsolutePath()+" ¿existe? "+ temp.exists());  
    mostrarEstado(fotos);  
    mostrarEstado(document);  
}
```

```
public static void mostrarEstado(File f) {  
    System.out.println(f.getAbsolutePath() + " ¿archivo? "+ f.isFile());  
    System.out.println(f.getAbsolutePath() + " ¿carpeta? "+ f.isDirectory());  
}
```

**EJERCICIO:** crear la carpeta "Temp" en la raíz "C:". Dentro, un archivo llamado "Documento.txt" (puede estar vacío) y una carpeta llamada "Fotos". Prueba el programa. Después de probarlo puedes eliminar algún elemento y volver a probar para ver la diferencia.

# Métodos: propiedades de ficheros

```
public static void main(String[] args) {  
    File documento = new File("C:/Temp/Documento.txt");  
    System.out.println(documento.getAbsolutePath());  
  
    long milisegundos = documento.lastModified();  
    Date fecha = new Date(milisegundos);  
  
    System.out.println("Última modificación (ms)    : " + milisegundos);  
    System.out.println("Última modificación (fecha): " + fecha);  
    System.out.println("Tamaño del archivo: " + documento.length());  
}
```

**EJERCICIO:** crea el archivo "Documento.txt" en la carpeta "C:\Temp". Primero deja el archivo vacío y ejecuta el programa. Luego, con un editor de texto, escribe cualquier cosa, guarda los cambios y vuelve a ejecutar el programa. Observa cómo el resultado es diferente. Como curiosidad, fíjate en el uso de la clase Date para poder mostrar la fecha en un formato legible.

# Métodos: gestión de ficheros I

```
public static void main(String[] args) {  
  
    File fotos = new File("C:/Temp/Fotos");  
    File doc = new File("C:/Temp/Documento.txt");  
  
    boolean mkdirFot = fotos.mkdir();  
  
    if (mkdirFot) {  
        System.out.println("Creada carpeta " + fotos.getName() + "? " + mkdirFot);  
    }  
    else {  
        boolean delCa = fotos.delete();  
        System.out.println("Borrada carpeta " + fotos.getName() + "? " + delCa);  
        boolean delAr = doc.delete();  
        System.out.println("Borrado archivo " + doc.getName() + "? " + delAr);  
    }  
}
```

**EJERCICIO:** primero asegúrate de que en la raíz de la unidad "C:" no hay ninguna carpeta llamada "Temp" y ejecute el programa. Todo fallará, ya que las rutas son incorrectas (no existe "Temp"). Luego, crea la carpeta "Temp" y en su interior crea un nuevo documento llamado "Documento.txt" (puede estar vacío). Ejecuta el programa y verás que se habrá creado una nueva carpeta llamada "Fotos". Si lo vuelves a ejecutar por tercera vez podrás comprobar que se habrá borrado.

# Métodos: gestión de ficheros II

```
public static void main(String[] args) {  
  
    File origenDir = new File("C:/Temp/Fotos");  
    File destinoDir = new File("C:/Temp/Media/Fotografies");  
    File origenDoc = new File("C:/Temp/Media/Fotografies/Document.txt");  
    File destinoDoc = new File("C:/Temp/Document.txt");  
  
    boolean res = origenDir.renameTo(destinoDir);  
    System.out.println("Se ha movido y renombrado la carpeta? " + res);  
    res = origenDoc.renameTo(destinoDoc);  
    System.out.println("Se ha movido el documento? " + res);  
}
```

**EJERCICIO:** Dentro de la carpeta "C:/Temp" crea una carpeta llamada "Media" y otra llamada "Fotos". Dentro de la carpeta "Fotos" crea dos documentos llamados "Documento.txt" y "Fotos.txt". Después de ejecutar el programa, observa como la carpeta "Fotos" se ha movido y ha cambiado de nombre, pero mantiene en su interior el archivo "Fotos.txt". El archivo "Documento.txt" se ha movido hasta la carpeta "Temp".

# Métodos: listado de ficheros

```
public static void main(String[] args) {  
  
    File dir = new File("C:/Temp");  
    File[] lista = dir.listFiles();  
    System.out.println("Contenido de " + dir.getAbsolutePath() + " :");  
  
    for (int i = 0; i < lista.length; i++) {  
        File f = lista[i];  
        if (f.isDirectory()) {  
            System.out.println("[DIR] " + f.getName());  
        } else {  
            System.out.println("[ARX] " + f.getName());  
        }  
    }  
}
```

**EJERCICIO:** Prueba este código. Antes de ejecutarlo crea una carpeta "Temp" en la raíz de la unidad "C:". Dentro crea o copia cualquier cantidad de carpetas o archivos.

# Referencia básica clase File

```
File f = new File  
("src/documentos/ejemplo.txt");
```

```
// evalúa si es un directorio  
f.isDirectory()
```

```
// evalúa si es n fichero  
f.isFile();
```

```
// crea el directorio correspondiente  
f.mkdir();
```

```
// crea todos los directorios si no están  
creados hasta llegar al indicado  
f.mkdirs();
```

```
// comprueba si existe  
f.exists();
```

```
// crea el fichero indicado  
f.create();
```

```
// borra el fichero indicado  
f.delete();
```

```
// lista los archivos del  
directorio indicado  
f.list();
```

```
// indica si se puede leer  
f.canRead();
```

```
// indica si se puede  
escribir  
f.canWrite();
```

```
// renombre el fichero  
indicado a un nombre dado  
f.renameTo();
```

# Lectura y escritura de ficheros

- Diferenciar entre:
  - Flujo de caracteres:
    - `File f = new File ("src/docs/ejemplo.txt");`
    - `FileReader fr = new FileReader(f);`
      - `BufferedReader br = new BufferedReader(fr);`
    - `FileWriter fr = new FileWriter(f);`
      - `BufferedWriter br = new BufferedWriter(fr);`
  - Flujo de datos binarios (+ complejidad)
    - Clases derivadas de `InputStream` / `OutputStream`.
      - `FileInputStream` (in/out)
      - `DataInputStream` (in/out)
      - `ObjectInputStream` (in/out)



# Lectura de ficheros (caracteres)

- 2 estrategias básicas:
  - Scanner
  - FileReader (+ BufferedReader)
- Scanner
  - Puede leer directamente múltiples formatos
  - Menor eficiencia
- FileReader (+ BufferedReader)
  - Lee “sólo” en formato String y debemos procesarlo
  - Necesita a BufferedReader para mejorar eficiencia
  - Y para permitir el método `.readLine()`; sino sólo sirve `.read()`;

# Escritura en ficheros (caracteres)

- 2 estrategias básicas:

- FileWriter
- PrintWriter

+ BufferedWriter

- FileWriter

```
File f = new File("C:\\Programas\\Unidad11\\Documento.txt");  
FileWriter writer = new FileWriter(f);  
writer.write("8");
```

- PrintWriter

```
File f = new File("C:\\Programas\\Unidad11\\Documento.txt");  
PrintWriter pw = new PrintWriter(f);  
pw.print("Lo que sea");
```

**CERRAR el flujo al acabar en lectura y escritura.  
En escritura es más crítico todavía.**

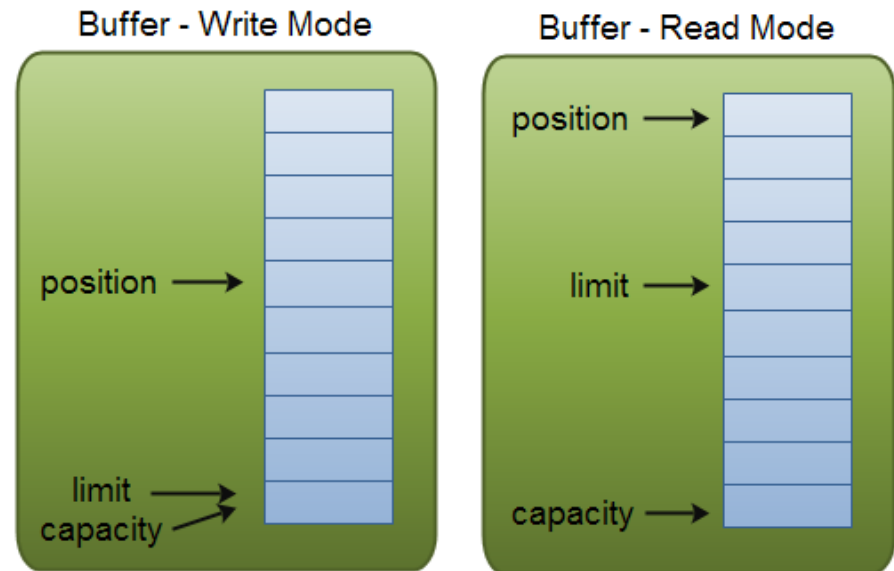
# Buffers para lectura y escritura

- Si usamos **FileReader** o **FileWriter**, cada lectura o escritura se hará físicamente en disco. Si hacemos muchas operaciones pequeñas, el proceso se hace costoso y lento, con muchos accesos a disco.
- Las clases **BufferedReader** y **BufferedWriter** añaden un *buffer* intermedio. Cuando leamos o escribamos, esta clase controlará los accesos a disco.

En **escritura**: se guardará los datos hasta que tenga bastantes datos como para hacer la escritura eficiente.

En **lectura**: la clase leerá muchos datos de golpe, aunque sólo nos dé los que hayamos pedido. En las siguientes lecturas nos dará lo que tiene almacenado, hasta que necesite leer otra vez.

Los **accesos a disco son más eficientes**. La diferencia se notará más cuanto mayor sea el fichero que queremos leer o escribir.



# Ejemplo de lectura I - Scanner

```
import java.io.File;
import java.util.Scanner;

class LeeFichero {

    public static final int NUM_VALORES = 10;

    public static void main(String [] arg){
        try{
            File archivo = new File ("enteros.txt");
            Scanner lector = new Scanner(archivo);

            for (int i = 0; i < NUM_VALORES; i++) {
                int valor = lector.nextInt();
                System.out.println("Valor leído: " + valor);
            }

            fr.close();
        }catch(Exception e){
            e.printStackTrace();
        }
    }
}
```

¿Problemas?

# Ejemplo de lectura I - Scanner

```
// Leemos el contenido del fichero
System.out.println("... Leemos el contenido del fichero ...");
Scanner s = new Scanner(fichero);

// Leemos linea a linea el fichero
while (s.hasNextLine()) {
    String linea = s.nextLine();    // Guardamos la linea en un String
    System.out.println(linea);     // Imprimimos la linea
}
```

```
// Leer números enteros en cada línea
while(s.hasNext()){
    int numero = leerArchivo.nextInt();
    // Procesar numero
}
```

## Diferencias next() y nextLine()

- **next()** sólo lee **1 palabra individual** (no están separados por espacios o saltos de línea).
- **nextLine()** lee **todo el texto que encuentre** (espacios incluidos) hasta el siguiente **salto de línea**.

## Referencia de métodos para Scanner

- hasNext()
- next()
- nextLine()
- nextInt()
- nextDouble()
- nextBoolean()

# Ejemplo de lectura II – FileReader

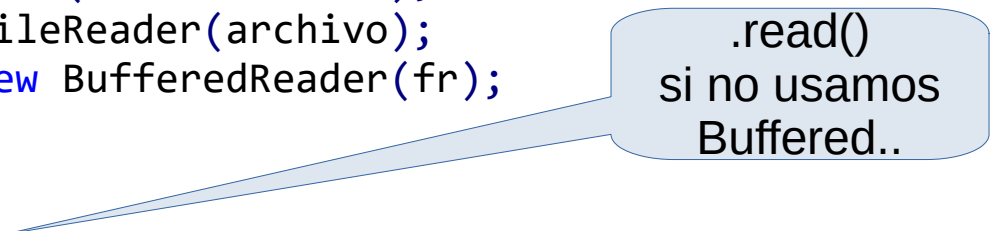
```
import java.io.File;

class LeeFichero {
    public static void main(String [] arg){
        try{
            File archivo = new File ("archivo.txt");
            FileReader fr = new FileReader(archivo);
            BufferedReader br = new BufferedReader(fr);

            String linea;

            while((linea=br.readLine())!=null)
                System.out.println(linea);

            fr.close();
        }catch(Exception e){
            e.printStackTrace();
        }
    }
}
```



.read()  
si no usamos  
Buffered..

# Ejemplo de escritura I - FileWriter

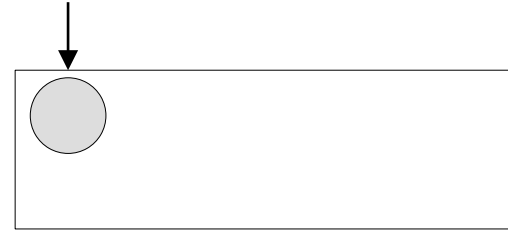
```
import java.io.*;

public class EscribeFichero{
    public static void main(String[] args) {
        try {
            File f = new File("Enteros.txt");
            FileWriter fw = new FileWriter(f);
            int valor = 1;
            for (int i = 1; i <= 20; i++) {
                fw.write("" + valor); // escribimos valor
                fw.write(" "); // escribimos espacio en blanco
                valor = valor * 2; // calculamos próximo valor
            }

            fw.write("\n"); // escribimos nueva línea
            fw.close(); // cerramos el FileWriter
            System.out.println("Fichero escrito correctamente");
        } catch (IOException e) {
            System.out.println("Error: " + e);
        }
    }
}
```

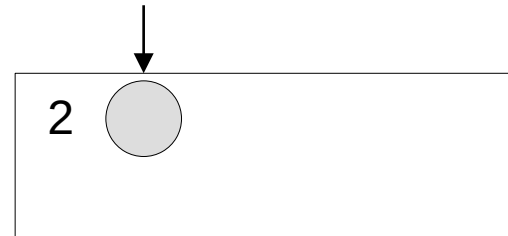
```
File f = new File("Enteros.txt");  
FileWriter fw = new FileWriter(f);
```

Puntero señala inicio



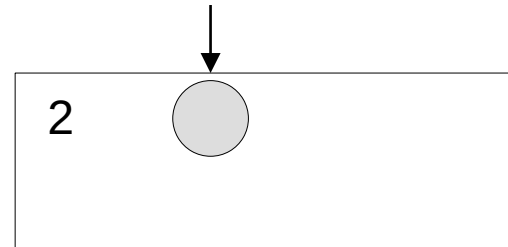
```
fw.write("2");
```

Escribimos "2". Puntero avanza hasta la siguiente posición



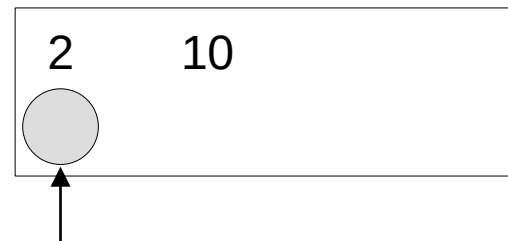
```
fw.write(" ");
```

Se escribe un espacio. Puntero avanza hasta la siguiente posición



```
fw.write("10\n");
```

Atención: usamos un "\n"





# Ejemplo de escritura II - PrintWriter

```
import java.io.*;

public class EscribeFichero{
    public static void main(String[] args){
        try{
            FileWriter fichero = new FileWriter("prueba.txt");
            PrintWriter pw = new PrintWriter(fichero);

            for (int i = 0; i < 10; i++){
                pw.println("Linea " + i);
            }
        } catch (Exception e) {
            e.printStackTrace();
        } finally {
            fichero.close();
        }
    }
}
```

Ambas clases sirven para escribir caracteres en ficheros, simplemente **PrintWriter** es una **mejora** de la clase **FileWriter** (comparten el mismo padre `java.io.Writer`) que nos permite utilizar métodos adicionales.

# Añadir información a fichero

## Añadir texto a un fichero existente

Siempre que escribamos en un fichero que ya existe, el fichero se sobrescribirá.

**PrintWriter** no tiene control sobre esto, pero sí la clase de la que hereda: **FileWriter**. En este caso debemos crear un **PrintWriter** a partir de un **FileWriter**.

Para añadir texto a un fichero en lugar de sobrescribirlo, indicaremos un segundo parámetro al constructor de **FileWriter**:

```
//Indicando true en el constructor de FileWriter, añadimos texto
File fichero = new File("fichero.txt");
FileWriter filewriter = new FileWriter(fichero, true);
PrintWriter escritor = new PrintWriter(filewriter);

//o sin el objeto File
FileWriter filewriter = new FileWriter("fichero.txt", true);
PrintWriter escritor = new PrintWriter(filewriter);
```

# Mejora de escritura con *Buffered...*

```
String[] lineas = { "Uno", "Dos", "Tres", "Cuatro", "Cinco", "Seis"};

FileWriter fichero = null;
BufferedWriter bw = null;

try {
    fichero = new FileWriter("ficheroescritura.txt");
    bw = new BufferedWriter(fichero);

    // Escribimos linea a linea en el fichero
    for (int i=0;i<lineas.length;i++) {
        bw.write(linea[i] + "\n");
    }

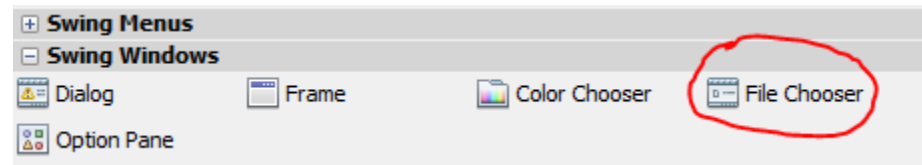
    fichero.close();
}
```

## Envoltura de objetos (Object wrapping)

```
PrintWriter writer = new PrintWriter(
    new BufferedWriter (
        new FileWriter("fichero.txt")));
```

# Componentes GUI Swing

- **JFileChooser**: muestra ventana seleccionar fichero.



```
int seleccion = jFileChooser1.showOpenDialog(this);
```

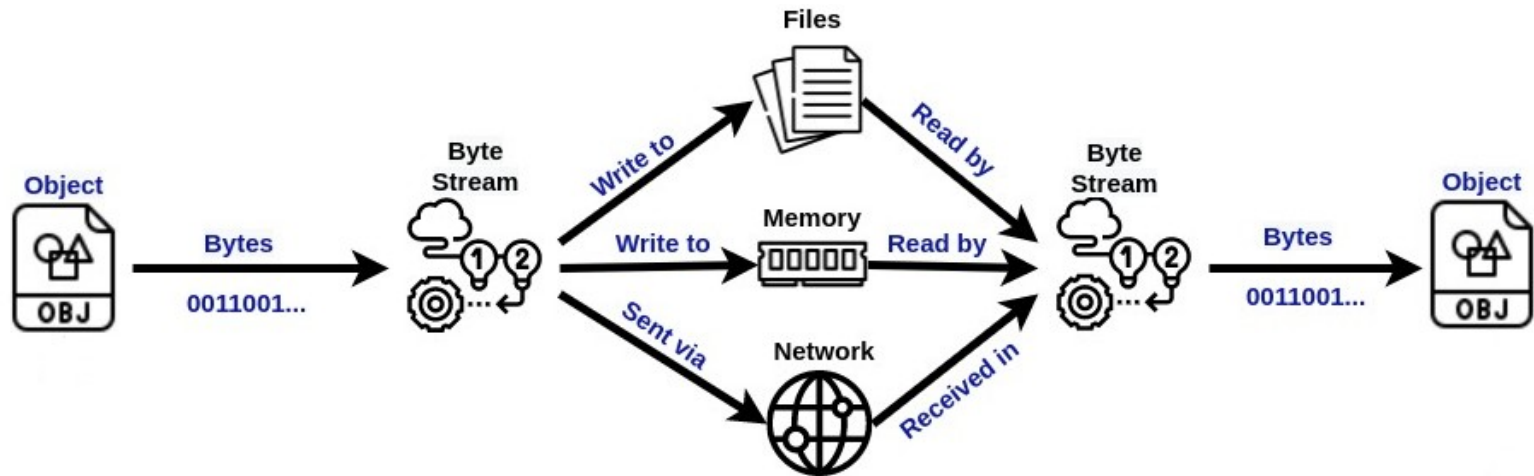
Devuelve una constante: ver tema 7.

```
File fichero = jFileChooser1.getSelectedFile();
```

`getSelectedFile()` devuelve un objeto de tipo File

# Serialización

- La serialización es el proceso por el cual un objeto se convierte en una secuencia de bytes que pueden ser almacenados en un archivo, y recuperados más adelante.
- Cuando se serializa un objeto se almacena la estructura de la clase y los datos contenidos.
- Para serializar y deserializar objetos se usa la interfaz `java.io.Serializable`



# Escritura de objetos en ficheros

```
File fichero = new File("fichero.bin");
FileOutputStream flujoFichero = new FileOutputStream(fichero);
ObjectOutputStream serializador = new ObjectOutputStream(flujoFichero);

Coche miCoche1 = new Coche();
Coche miCoche2 = new Coche();

// Escribir objetos en el fichero
// Puedo almacenar objetos String
serializador.writeObject("Guardo 2 objetos Coche");

// u objetos tipo Coche
serializador.writeObject(miCoche1);
serializador.writeObject(miCoche2);
. . .
//Cerrar los flujos de salida
serializador.close();
```

**Reflexiona:** ¿cómo podríamos escribir todos los coches almacenados en un ArrayList?

# Lectura de objetos de ficheros

El proceso de lectura desde fichero es paralelo al de escritura. Debo usar los flujos anteriores, pero en este caso deben ser de lectura:

Flujo de entrada desde fichero: `FileInputStream`

Flujo de entrada de objetos: `ObjectInputStream`

Método de `ObjectInputStream` para lectura de objetos: `readObject()`

```
File fichero = new File("fichero.bin");
FileInputStream flujoFichero = new FileInputStream(fichero);
ObjectInputStream deserializador = new ObjectInputStream(flujoFichero);

//Leer el objeto del fichero (en el mismo orden !!)
//Leo el objeto tipo String
String cadena = (String) flujoObjetos.readObject();

//Leo los objetos tipo Coche
Coche cocheLeido1 = (Coche) deserializador.readObject();
Coche cocheLeido2 = (Coche) deserializador.readObject();

//Finalmente se cierran los flujos de entrada
deserializador.close();
```

# Parte II

## Acceso a bases de datos



# Persistencia con bases de datos

- Más potente que el uso de ficheros, pero su uso es más complejo.
- Recomendado en la mayoría de ocasiones.
- Gestión de los datos principales del programa.
- Cuidado con el uso del SGBD y su arquitectura.
- Suelen requerir uso de estrategias más avanzadas: mapeo objeto-relacional, etcétera...
- Se trata en la fase de diseño de una aplicación.

# Acceso a bases de datos

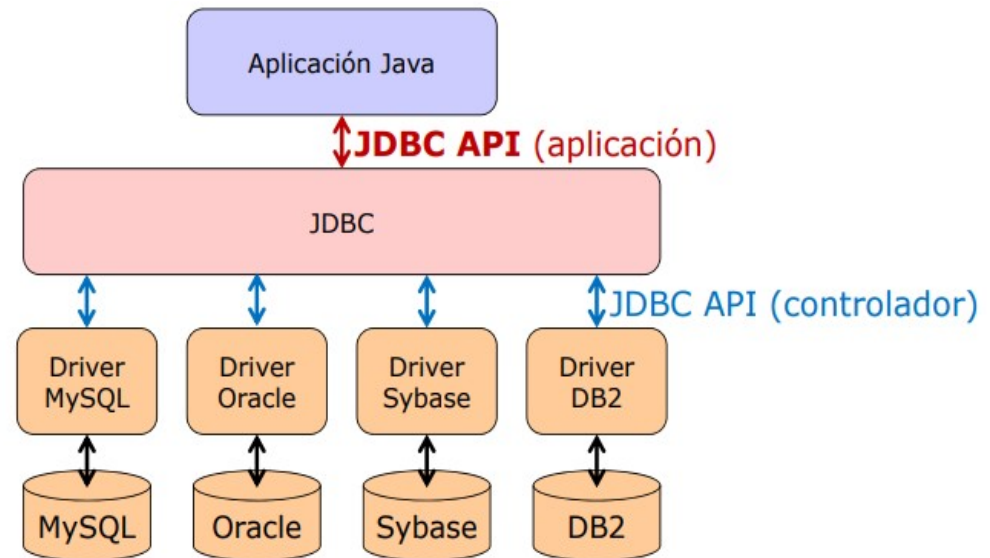
- La librería Java Database Connectivity (JDBC) proporciona acceso a BD desde Java.
- 2 paquetes: `java.sql` y `javax.sql`
- Para trabajar con un SGBD se requiere un driver concreto que haga de intermediario entre JDBC y el SGBD.
- ¿Conclusión? A priori un mismo programa que funcione sobre Oracle no lo hará sobre MySQL...

# Acceso a bases de datos

JDBC es un API Java que hace posible interactuar con bases de datos.

**Conector** (driver): conjunto de clases encargadas de implementar las interfaces del API y acceder a la base de datos. Imprescindible para poder conectarse a una base de datos e interactuar con ella.

**JDBC** consulta al driver, que oculta los detalles específicos de cada base de datos, de modo que el programador se preocupa sólo de su aplicación (**ojo con el lenguaje SQL...**)



El conector lo proporciona el fabricante de la base de datos, o bien un tercero.

El código de nuestra aplicación **no depende del driver**, puesto que trabajamos contra los paquetes `java.sql` y `javax.sql`, es decir, el API JDBC.

# Conexión con MySQL

- Realizar conexión:

```
Connection conBD = null;
String servidor = "jdbc:mysql://localhost:3306/";
String basedatos = "personas";
String user = "root";
String password = "123456";

Try {

    conBD = DriverManager.getConnection(servidor + basedatos, user, password);

} catch (SQLException error) {
    System.out.println("Error al conectar con MySQL: " + error.getMessage());
}
```

- Cerrar conexión:

```
conBD.close();
```

# Consulta simple en “bloque”

- Preparar y ejecutar consultas (DML)

//Objetos necesarios para gestionar la consulta a la BD

Statement **stm** = **null**;

ResultSet **rs** = **null**;

**try** {

**stm** = **conBD.createStatement()**;

**rs** = **stm.executeQuery("Select nombre from personas")**;

//Recorremos todos los registros devueltos en el ResultSet

**while** (**rs.next()**) {

**System.out.println("Nombre: " + rs.getString(1))**;

}

} **catch** (SQLException **error**) {

**System.out.println("Error en consulta: " + error.getMessage())**;

} **finally** {

**rs.close()**;

**stm.close()**;

**conBD.close()**;

}



Imprescindible

# Clase Statement

- **ResultSet executeQuery(String sql):** ejecuta la sentencia SQL indicada (de tipo SELECT). Devuelve un objeto ResultSet con los datos resultado de la consulta.

```
ResultSet rs = stmt.executeQuery("SELECT * FROM vendedores");
```

- **int executeUpdate(String sql):** ejecuta la sentencia SQL indicada (DML de tipo INSERT, UPDATE o DELETE). Devuelve el número de registros que han sido insertados, modificados o eliminados.

```
int nr = stmt.executeUpdate("INSERT INTO personas VALUES (1, 'Paco');")
```

# Clase ResultSet

Es una estructura de datos en forma de tabla con registros (filas) que podemos recorrer para acceder a la información de sus campos (columnas).

**boolean next():** mueve el cursor al siguiente registro. Devuelve true si fue posible o false en caso contrario (final de la tabla).

Método para obtener los datos del registro actual:

**String getString(int columnIndex):** devuelve un String de la columna indicada por su nombre. La primera columna es la 1, no la 0.

Otros métodos análogos para obtener tipo int, long, float, double, boolean, Date, Time, Array, etc:

```
int getInt(int columnIndex)
double getDouble(int columnIndex)
boolean getBoolean(int columnIndex)
Date getDate(int columnIndex)
etc...
```